

程序设计实习实验班 2026 期末考试题解

A. 欧氏距离平方和查询

Authored by 李雷思问, Prepared by 李雷思问

不妨设数据点依次为 $x_1, x_2, \dots, x_n$, 由 $(a - b)^2 = a^2 - 2ab + b^2$, 考虑令:

$$\bullet A_k = \sum_{i=1}^n x_{i,k}^2, B_k = \sum_{i=1}^n x_{i,k}$$

此时对于一个查询点 y , 他的误差平方和即为:

$$\begin{aligned} \sum_{i=1}^n \sum_{k=1}^d (x_{i,k} - y_k)^2 &= \sum_{i=1}^n \sum_{k=1}^d (x_{i,k}^2 - 2x_{i,k}y_k + y_k^2) = \sum_{i=1}^n \sum_{k=1}^d (x_{i,k}^2 - 2x_{i,k}y_k + y_k^2) \\ &= \sum_{k=1}^d \left(\sum_{i=1}^n x_{i,k}^2 - \sum_{i=1}^n 2x_{i,k}y_k + \sum_{i=1}^n y_k^2 \right) = \sum_{k=1}^d (A_k - 2B_k y_k + n y_k^2) \end{aligned}$$

预处理出所有 A_k, B_k 即可, 复杂度 $O((n + q) \cdot d)$ 。

B. 坐标轴上的曼哈顿匹配

Authored by 葛致远, Prepared by 李雷思问

对于两点 u, v , 他们分别处于第 i, j 条坐标轴, 而下标分别为 x, y , 此时他们的曼哈顿距离为:

- 如果 $i \neq j$, 或者 x, y 不同号, 则为 $|x| + |y|$ 。
- 只有当 $i = j$ 且 x, y 同号时, 为 $|x| + |y| - 2 \min(|x|, |y|)$ 。

于是以 $S = \sum_{i=1}^n \sum_{k=1}^d |x_{i,k}|$ 为基准, 只有将处于相同坐标轴且下标同号的点匹配起来才能使得匹配代价减小, 并且减小的值为坐标绝对值较小的的两倍。

于是根据所处的坐标轴和坐标的符号, 可以将点分为 $2d$ 类, 而对于每一类, 重复如下过程:

- 如果该类中至少还有两个未匹配的点, 则将坐标绝对值最大和次大的匹配起来。

其他没有匹配的点任意匹配即可, 根据之前的讨论易证这是最优的。

复杂度 $O((n + q) \cdot d)$, 如果输入是直接给出所处的坐标轴和下标甚至可以做到 $O(n + q)$ 。

C. 图的重染色问题

Authored by 方心童, Prepared by 方心童

我们尝试按下标 i 从小到大的顺序依次将 i 的颜色变成 b_i 。

设当尝试将 i 的颜色变成 b_i 时, 图的染色为 c , 对于 i 的所有邻居 j 有:

- 如果 $j < i$, 一定有 $c_j = b_j$, 而 $b_j \neq b_i$, 故 $c_j \neq b_i$, 不需要改动 j 的颜色。
- 如果 $j > i$, 由于 $k = d + 2$, 必定存在某个颜色 x , 使得 $x \neq b_i$ 且该颜色不为与 j 相邻的点的颜色, 将 j 的颜色改为 x 即可。

做完如上操作后，将 i 的颜色改为 b_i 即可，容易证明该调整总次数为 $n + m$ ，时间复杂度草率实现即可 $O((n + m)^2)$ 。

D. 合力平衡

Authored by 刘鹏飞, Prepared by 刘鹏飞

可以直接随机化：

- 直接给所有二维向量随机一个方向，可以证明合力大小的平方期望为 $\sum_{i=1}^n (x_i^2 + y_i^2)$ 。
- 直接使用 Union Bound 草率分析即至少有 $\frac{1}{6}$ 的概率成功，于是重复随机方向，直到合力大小的平方满足题目要求再输出即可。

也有一种直接贪心的正确做法：

- 考虑前 $i - 1$ 个向量得到的合力为 $(\bar{x}, \bar{y})$ ，则当加入第 i 个二维向量时，只关注 $(\bar{x} + x_i, \bar{y} + y_i)$ 和 $(\bar{x} - x_i, \bar{y} + y_i)$ 的哪一个大小更小，取更小的那一个作为前 i 个向量得到的合力。
- 可以证明这样贪心得到的最终的 S 满足 $S \leq \sum_{i=1}^n (x_i^2 + y_i^2)$ 。

E. 最小半包围圆 2-近似

Authored by 方心童, Prepared by 方心童

我们先给出最小包围圆的 2-近似算法：

- 直接任意选取一个数据点 o 作为圆心，计算所有其他数据点到该点的最大距离作为半径即可。
- 这是由于如果存在一个半径为 r 的小圆覆盖了 o ，则以 o 为圆心的半径为 $2r$ 的大圆一定能整个覆盖住小圆。

而该数据点集的最小半包围圆至少覆盖了该数据点集内一半的点，因此我们任意抽取该数据点集内的一个点，有至少 $\frac{1}{2}$ 的概率处于该点集的最小半包围圆内！

于是最小半包围圆的 2-近似算法为：

- 随机抽取一个数据点 o 作为圆心，计算其到所有其他数据点的距离，取中位数作为半径即可。可以发现如果 o 在真正的最小半包围圆内，则类似之前的分析，我们给出了一个最小半包围圆的 2-近似。
- 于是重复上述采样 T 次并取最优，就有至少 $1 - (\frac{1}{2})^T$ 的概率正确。

总复杂度 $O(Tn)$ ，其中 $T = \log \frac{1}{\delta}$ ，而 δ 为失败概率（中位数可以线性求，如果直接排序为 $O(Tn \log n)$ ，也能通过）。

F. 最小半包围圆 $(1 + \epsilon)$ 近似

Authored by 葛致远, Prepared by 方心童

考虑先运行最小半包围圆的 2-近似算法，则可以得到一个 R ，使得答案半径在区间 $[R, 2R]$ 内。

而如果我们将空间划分成 $l \times l$ 的小正方形，将所有数据点离散化到格点上，且要求圆心在格点上，此时有：

- 则当 r 为原问题的合法答案半径时，离散化后的问题一定存在一个圆半径为 $r + \sqrt{2}l$ 的答案（圆心和数据点分别至多偏离 $\frac{\sqrt{2}}{2}l$ ）。
- 离散化后的解在半径增加 $\frac{\sqrt{2}}{2}l$ 后一定能覆盖原数据点集（每个数据点至多偏离 $\frac{\sqrt{2}}{2}l$ ）。

于是取 $l = \frac{\sqrt{2}}{3}\varepsilon R$, 则在离散化后的问题上取得的圆心可以作为原问题的圆心, 并得出一个半径至多比答案大 εR 的解。

此时发现即使是半径为 $2R$ 的圆, 也至多只能覆盖 $O(\frac{1}{\varepsilon^2})$ 个格点, 于是:

- 随机抽取一个数据点 o , 设其被离散化到了格点 o' 上, 我们强制要求我们找出的半包围圆覆盖格点 o' , 此时合法的候选圆心只会有 $O(\frac{1}{\varepsilon^2})$ 个 (这是由于候选圆心必须在格点上, 而又要求以该格点为圆心的半径为 $2R$ 的圆要能覆盖格点 o')。而每个合法的圆心, 即使当半径取到 $2R$ 时也只能覆盖 $O(\frac{1}{\varepsilon^2})$ 个格点。
- 于是暴力枚举候选圆心, 再暴力扩大半径, 直到成为一个合法的半包围圆或半径已经超出 $2R$, 而距离圆心距离不超 $2R$ 的点的顺序可以预处理, 总复杂度为 $O(\frac{1}{\varepsilon^4})$ 。
- 可以发现当 o 的确位于最小的半包围圆内时, 该算法一定能给出正确的 $(1 + \varepsilon)$ 近似。于是重复上述过程 T 次并取最优, 就有至少 $1 - (\frac{1}{2})^T$ 的概率正确。

总复杂度 $O(T(n + \frac{1}{\varepsilon^4}))$, 其中 $T = \log \frac{1}{\delta}$, 而 δ 为失败概率。

G. 二维欧氏空间范围点数查询 (加强版)

Authored by 葛致远, Prepared by 葛致远

考虑做半径为 1 的圆的外接 k 边形, 再做该 k 边形的外接圆, 则该圆的半径为 $\sec \frac{\pi}{n} = 1 + \frac{\pi^2}{2n^2} + O(\frac{1}{n^4})$ 。

于是取 $k = O(\frac{1}{\sqrt{\varepsilon}})$ 时, 如果我们可以精确数其外接 k 边形内的点数即可解决本题。而数一个 k 边形内的点数可以拆成 k 组二维偏序, 于是总复杂度为 $O(nk \log n) = O(\frac{n \log n}{\sqrt{\varepsilon}})$ 。

看起来可能有点脑经急转弯, 事实也的确如此。但实际上本题给出了随机旋转的一个很好的解释。

在二维平面上, 我们类似这种“正 k 边形”的思想, 将角度 k 等分, 可以发现对于任意一条长度为 1 线段, 其在某一个角度上的投影至少是 $\cos \frac{\pi}{n} = 1 - \frac{\pi^2}{2n^2} + O(\frac{1}{n^4})$ 。

因此我们只需要找 $O(\frac{1}{\sqrt{\varepsilon}})$ 个角度, 就可以给出任意一个线段长度的 $(1 \pm \varepsilon)$ 近似。而这有什么用呢? 我们可以回顾

作业二十二: 欧氏点集直径的数据流算法。

我们只要找到 $O(\frac{1}{\sqrt{\varepsilon}})$ 个角度并做投影, 就可以将问题归约到一维上! 而一维问题上有空间 $O(\frac{1}{\varepsilon} \log \Delta)$ 的算法:

- 依次取尺度 $r = 1, 2, 4, 8, \dots, \Delta$, 我们猜测答案在 $[r, 2r)$ 内:
 - 将所有点坐标对 $5r$ 取模, 映射到一个长度为 $5r$ 的环上。
 - 将该环按 $\min(\frac{\varepsilon}{2}, 0.1)r$ 的长度划段, 并将每个点离散化到段中心 (通过计数器判断最终是否还有点)。
 - 找该环上最短的覆盖作为估计的直径。如果答案的确处于 $[r, 2r)$ 内, 则估计的结果必定在 $[0.9, 2.1r)$ 内。于是我们当且仅当估计的结果在 $[0.9r, 2.1r)$ 内时才返回估计的直径, 否则返回 0。
- 对所有尺度返回的估计结果取最大, 作为直径的估计结果即可。这是由于:
 - 设实际直径 x 属于 $[r_0, 2r_0)$ 内, 则在 $[R, 2R)$ 的尺度区间上, 估计的结果不会超 $x + \frac{R}{10}$, 于是当 $R \geq 4r_0$ 时, 有 $x + \frac{R}{10} \leq 2r_0 + \frac{R}{10} < 0.9R$, 该尺度不会给出返回估计结果。
 - 而在 $[r_0, 2r_0)$ 的尺度下, 一定会给出一个误差不超 $\frac{\varepsilon}{2}r$ 的估计。而在 $[2r_0, 4r_0)$ 即以下的尺度下, 不会给出超出答案 εr 以上的估计, 于是一定能得到答案误差不超 εr 的估计。

综上我们给出了没有失败概率, 且空间复杂度 $O(\frac{1}{\sqrt{\varepsilon}} \times \frac{1}{\varepsilon} \log \Delta)$ 的算法, 空间远比课上讲的算法优秀 (代价是时间复杂度增加)。

而在 d 维下同样可以证明，只需要选取 $O(\varepsilon^{-\frac{d-1}{2}})$ 个投影即可满足需求，于是在 d 维下可以做到 $O(\varepsilon^{-\frac{d+1}{2}} \log \Delta)$ 的总空间复杂度。

Ex. 最小 k-包围圆 (1 + eps) 近似

Authored by 葛致远

最小 k-包围圆，指的是半径最小的，能包含至少 k 个数据点的圆。此处我们给出最小 k-包围圆的 $O(n \log \Delta + \frac{n}{k} \cdot \frac{1}{\varepsilon^3} \log \frac{1}{\varepsilon})$ 的算法。

我们先在 $O(n \log \Delta)$ 内给出答案的常数倍近似。直接二分半径 R ，然后：

- 按 $\frac{R}{10}$ 画格子，强制钦定圆心在格点上，则每个数据点只会至多在 400 个候选圆心对应的半径为 R 的圆内，利用哈希表即可在 $400n$ 的时间内更新所有候选圆心能覆盖的数据点数，查看其中是否有覆盖了至少 k 个数据点的即可。

不难发现单次时间复杂度 $O(n)$ ，于是在总 $O(n \log \Delta)$ 的时间内即给出了答案的 1.5-近似（实际上应该更紧，但不重要）。

不妨设我们讲答案半径确定在了区间 $[R, 1.5R]$ 内，此时再：

- 按 R 画“大格子”，强制钦定圆心在格点上，找出那些半径取 $2.5R$ 时能覆盖至少 k 个点的圆心，则这样的“可行大圆心”只有 $O(\frac{n}{k})$ 个（每个数据点只会被附近不超 100 个圆心在半径取 $2R$ 时覆盖到）。

再按 $\min(\frac{\sqrt{2}}{6\varepsilon}, 0.1)R$ 画“小格子”，将所有数据点离散化到小格子的格点上，并要求圆心落在小格子的格点上，根据之前的结论，此问题得出的圆心可以作为原问题的圆心，至多只会导致半径增大 $\frac{\varepsilon}{2}R$ 。

而对于一个小格子的格点作为圆心，半径取 $1.5R$ 时，如果其能覆盖离散化后的至少 k 个点。则对于任意一个与该小格子格点距离不超 $0.8R$ 的点，取该点为圆心，以 $2.5R$ 为半径画圆，一定能覆盖原点集中的至少 k 个点（每个点离散化前后至多偏离 $0.1R$ ，圆心偏离 $0.8R$ ，因此总偏离至多为 $0.9R$ ，一定能覆盖原来的半径为 $1.5R$ 的圆覆盖的所有点）。

而任取一个点，其距离 $0.8R$ 内一定存在“大格子”的格点，于是任意一个可能成为答案圆心的小格子格点，其附近一定有一个“可行大圆心”。

而“可行大圆心”只有 $O(\frac{n}{k})$ 个，每个“可行大圆心”距离 $0.8R$ 内的小格子格点只有 $O(\frac{1}{\varepsilon^2})$ 个，于是可能成为答案圆心的小格子格点只有 $O(\frac{n}{k} \cdot \frac{1}{\varepsilon^2})$ 个。

对于每个小格子格点，在 $[R, 1.5R]$ 内二分答案，并通过前缀和加快计算圆内的点数（拆成每一行的区间和），即可做到 $O(\frac{1}{\varepsilon} \log \frac{1}{\varepsilon})$ 。

故总复杂度为 $O(n \log \Delta + \frac{n}{k} \cdot \frac{1}{\varepsilon^3} \log \frac{1}{\varepsilon})$ 。

值得一提的是，这似乎比 [Fast Algorithms for Computing the Smallest k-Enclosing Circle](#) 一文中给出的算法少一个 $\log \frac{1}{\varepsilon}$ ，虽然我猜也没什么人研究这个东西就是了。